

LazyTick: Lazy and Efficient Management of Job Release in Real-Time Operating Systems

ANONYMOUS AUTHOR(S)

Releasing jobs and performing scheduling decisions in real-time operating systems (RTOSes) is often realized within tick interrupts. In each tick interrupt, a set of tasks that are waiting to release new jobs, namely the waiting set, is inspected to determine the jobs that should be released at this tick. Such a waiting set is sorted whenever a job has finished, by which the release process can be efficiently achieved. However, the overhead of sorting can vary vastly depending on the task set, which has to be taken into account in the worst-case timing analysis. Moreover, the tick interrupt in common practices is backed by a single hardware timer that is configured to trigger interrupts, either with a fixed period or reconfigured to the next release time (so-called one-shot timer). Since not necessarily at every interrupt a job will be released, several tick interrupts might be redundant. For the one-shot timers, the reconfiguration during runtime also incurs overheads at a variable interval. To reduce such variability and amount of overhead, in this work, we propose *LazyTick* which partitions the task set and distributes the subsets over multiple timers. Specifically, we propose two job release procedures with constant operations overhead—one for harmonic task sets and one for non-harmonic task sets. We implemented the support for multiple hardware timers in FreeRTOS and conducted intensive experimental evaluations. The evaluation shows that *LazyTick* can reduce the variability in overhead by up to $\approx 5\times$ in peak and $\approx 3\times$ on average in comparison to the default implementation. Additionally, the combined overhead of the job release process is reduced by up to $\approx 6.1\times$ in peak and $\approx 3.6\times$ on average.

Additional Key Words and Phrases: Tick Interrupts, Job Release, Hardware Timer, RTOS, Harmonic Task Sets

ACM Reference Format:

Anonymous Author(s). 2025. **LazyTick**: Lazy and Efficient Management of Job Release in Real-Time Operating Systems. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 25 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Real-time systems play an indispensable role in many critical applications, e.g., aerospace, automotive, medical, and industrial control systems, especially when timing guarantees are required. Specifically, the realization of real-time systems is often supported by real-time operating system (RTOSes) which process data and events under timing constraints. Therefore, improving the efficiency of RTOSes is beneficial in many design aspects [12, 17], such as processor utilization, schedulability and energy efficiency.

The execution of tasks in RTOSes involves both the scheduling policy and the process of releasing task instances (i.e., job releases). While the scheduling policy solely decides which of the available jobs should be executed at a given time, three major operations are performed during the job release process: 1) the tick counter used to keep the internal time is updated, 2) all jobs that have to be released at that tick have to be determined and inserted into the global ready queue, and 3) a scheduling decision has to be conducted. This process necessitates computational efforts and

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

may incur non-negligible overheads, which can be summarized into two main causes, namely 1) managing waiting times of tasks and 2) releasing the correct jobs during a given tick interrupt.

To manage waiting times of a task set, a housekeeping data structure is commonly used to keep track of the task specific waiting times, namely a *waiting set*¹. The organization of the waiting set poses a trade-off between adding complexity to the end of a job execution or the interrupt handler where new jobs are released. Keeping the waiting set sorted necessitates a sorted insertion into the waiting set whenever a job has finished. As this process has to be done for every job execution, the cost to insert jobs into the waiting set must be accounted for in the execution time of the job itself. Such overhead is often assumed as part of the worst-case execution time (WCET) in the literature to facilitate the schedulability analysis, and the variability and potential cost of sorted insertion may contribute to a much higher WCET. Although keeping no order in the waiting set may avoid the cost incurred by this process, the interrupt handler then has to inspect every task in each tick. This solution only shifts the overhead to the interrupt context, which is also problematic since interrupts stall the whole system.

Commonly, the *tick interrupt* is either realized as a periodic timer interrupt, as in FreeRTOS [24] and RTEMS [23], or realized as a one-shot timer interrupt, as it is used in LITMUS^{RT} [2] to handle the job release process. A periodic timer interrupt invokes the job release process at a fixed interval, while an implementation with a one-shot timer always sets the interrupt interval directly to the next release time of any job. The realization of periodic timer interrupts has no timer management overhead during runtime and occurs with a predictable pattern. However, interrupts need to be invoked at every job release tick, and there may be redundant interrupts where no job is actually released. For a one-shot implementation, the reconfiguration happens at runtime and results in an interrupt that occurs at a variable interval. Consequently, to reduce the interrupt overhead, it is not only important to reduce the overhead of managing the waiting set, but also to reduce the number of invoked tick interrupts to a minimum.

Conventionally, RTOSes make use of a single timer that is responsible for the job release process of the whole task set. However, modern processors (even microprocessors), such as the ESP32 family from Espressif, the MSP430 family from Texas Instruments, and the STM32Fx series from STMicroelectronics, often contain multiple hardware timers, that can be configured to tick independently at different interval lengths. In more domain-specific contexts, such as automotive systems, microcontrollers like the Infineon TC39x series and the TC1796 also offer multiple timers. The latter has been exploited by SLOTH on Time [12] to speed up the task dispatching while reducing its memory footprint. The proposed strategy is to map one timer to time-triggered event, and in case the number of timers is not sufficient, the mapping falls back to multiplexing support with a hardware-oriented remedy. While SLOTH on Time demonstrates the extreme of fully offloading the dispatcher to timers—requiring a large number of timer resources—our focus is different. We aim to integrate multiple hardware timers into standard RTOS behavior, using them efficiently to reduce interrupt and scheduling overhead without relying on an abundance of timer units.

Our Contributions: In this work, we consider *when* and *how* to efficiently manage the waiting set for periodic task systems, which are prevalent industry practices, e.g., 82% of the investigated systems adhering to periodic task activation in a recent empirical study [1]. We identify two factors as the main causes for the overheads for managing the waiting set: 1) the number of tick interrupts alongside their complexity and 2) the efficiency of delaying periodic tasks until they release a new

¹To handle recurring events such as periodic tasks, this type of housekeeping data structure is often referred to as a “waiting queue” [25]. Instead, we use the term “waiting set” to generalize the concept and avoid implying that a queue data structure must be used in the implementation.

job. To minimize the aforementioned overheads, we introduce *LazyTick*², an efficient job release mechanism that exploits multiple timers in RTOSes to minimize the overheads for handling the job releases of periodic real-time tasks. In a nutshell, we make the following contributions:

- A mixed-integer quadratically-constrained program (MIQCP) that takes a set of periodic tasks as input and constructs a partition thereof alongside a mapping of each partition to a separate hardware timer, with the objective of maximizing interrupt intervals, thereby minimizing the total number of tick interrupts (see Sec. 5).
- A job release procedure that realizes a constant-time overhead for both the insertion of delayed tasks and release of new jobs. For harmonic task sets, we exploit the static release order of jobs to eliminate the need to perform any sorting. For non-harmonic task sets, we employ a circular buffer whose size is bounded by the maximal multiple of the timer period for a given subset of tasks (see Sec. 6).
- An implementation of *LazyTick* in FreeRTOS alongside empirical evaluation of overheads caused by tick interrupts, insertion of delayed tasks, and retrieval of new jobs on a real-world embedded system (see Sec. 7). We plan to release the source code.

2 Related Work

Improving the efficiency of RTOSes is beneficial in many design aspects, such as processor utilization, schedulability and energy efficiency. Among these, task management—particularly the handling of the ready queue and the waiting set—has attracted notable attention in prior research. One notable line investigates efficient implementations of the Earliest Deadline First (EDF) scheduler. Since EDF requires dynamically prioritizing tasks in the ready queue based on their absolute deadlines, the associated runtime overhead is non-negligible. Notably, managing the ready queue in EDF shares similarities with managing the waiting set, especially under earliest release first behavior. In this work, we focus specifically on the management of the waiting set. For completeness, we categorize related work into two groups based on the target of task management: techniques addressing the ready queue, and those focusing on the waiting set.

The management of the ready queue. To avoid runtime overhead, this has been done elegantly by employing a priority table to realize fixed-priority scheduling in most RTOSes. As the priority of a task under EDF changes over time, implementing EDF efficiently requires an efficient management of the ready queue. For example, Mok [18] proposes to represent the ready queue with a “heap of heaps” data structure. Buttazzo and Gai leverage the Implicit Circular Timer’s Overflow Handler (ICTOH) proposed in [7] to implement an efficient EDF, which has been integrated into the ERIKA Enterprise kernel [5]. Brandenburg and Anderson propose to utilize binomial heaps [3].

Specifically, under FreeRTOS, Oliveira and Lima evaluate the efficiency of using min-heap for the ready queue and both ready-queue implementations as linked-lists that were introduced by Pathan [21, 22], on an ARM-M4 architecture [20].

The management of the waiting set. The recurrent tasks are prioritized according to the release time of their next to-be-released jobs. Therefore, the priority of a task in the waiting set changes over time, depending on the release time of the next to-be-released job. If sufficient memory is available, Varghese and Lauck [27] propose a timing wheel, which realizes *bucket sort* in a hierarchical manner. Short [25] extends the use of a timing wheel for representing the ready queue, termed a deadline wheel, and proposes alternative implementations for both the waiting set and the ready

²The term *Lazy* refers to our method extending the periodic tick interrupt intervals as far as possible to minimize of the number of tick interrupts; *Efficient Management of Job Release*, refers to the efficient design of the waiting set which reduces the work per tick interrupt.

queues, based on digital search trees (DST). Carlini and Buttazzo propose the ICTOH method [7] to represent time in a circular manner. Under FreeRTOS, Hagens and Chen assess the efficiency of various data structures, such as Red-Black trees, on the Xtensa LX7 MCU [11].

Hofer et al. propose SLOTH on Time, exploiting multiple timer cells available in the hardware to facilitate different housekeeping operations [12]. Their approach provides a solution with very little overhead, when sufficient timers are available to directly bind housekeeping operations of task releases to separate timers. However, our method deals with the case where there are fewer available timers than tasks. This is the scenario where SLOTH falls back to multiplexing timers, for which *LazyTick* offers a more systematic and scalable solution. Our waiting set design and coalesced tick scheduling are tailored for this fallback scenario. Given the distinct assumptions and system models, we consider the two approaches to be orthogonal.

Summary. In this work, we explore both *when* and *how* to manage the waiting set in an RTOS in this paper. While prior studies above [5, 11, 12, 22, 25, 27] focus solely on the mechanisms for managing the waiting set, we additionally address the timing of tick interrupts and investigate how the use of multiple hardware timers can enhance the efficiency of the waiting set management. As pointed by Colin and Puaut [8], the static WCET analysis of the ready queue in an RTOS can be made less pessimistic if prior knowledge of task repartitions is available. We believe the same principle applies to the waiting set. To the best of our knowledge, this work is the first to leverage properties of a given task set to tailor the management of the waiting set, while making use of available hardware timers. While our approach mapping multiple periodic tasks to single periodic timers shares a similar spirit as the approach by Khenfri et al. [13] where multiple periodic runnables are mapped to single periodic tasks, we specifically focus on exploiting such mappings in the context of an RTOS. Our work not only improves RTOS efficiency, but also has the potential to reduce pessimism in worst-case timing analysis by minimizing overhead-related variability.

3 System Model

In this work, we consider an RTOS where the job release process is triggered during a tick interrupt. These interrupts are backed by either a periodic timer or a one-shot timer and occur at integer multiples of a system tick interval, e.g., multiples of 1 ms. From the hardware perspective, we assume a single processor with multiple hardware timer sources. Given a minimal granularity of the system tick interval P , we assume to be able to program hardware timers to trigger interrupts at intervals of $s \cdot P$, where $s \in \mathbb{N}$. Two hardware facilities can be utilized to achieve these interrupts: First, a timer can be directly programmed to the given tick interval of $s \cdot P$ and furthermore be programmed to trigger an interrupt at every increase of the timer. Second, the timer can be programmed to not trigger interrupts on every increase of the timer, but rather use a compare register and only trigger interrupts at every s ticks of the timer. Specifically, we consider a set of m configurable timers $\mathbb{P} = \{\rho_1, \dots, \rho_m\}$, where a timer ρ_i periodically triggers an interrupt each P_i time units.

Generally, we consider a hard real-time task set $\mathbb{T} = \{\tau_1, \dots, \tau_n\}$ of n periodic tasks that should be executed on the system. We denote a periodic task by $\tau_i = (C_i, T_i, D_i)$ with C_i being the WCET, T_i the period of the task, and D_i the relative deadline of the task³, where the first job is released at time zero. The set of all task periods is given by $\mathcal{T}$ and the hyperperiod H of a task set $\mathbb{T}$ is defined

³As job arrivals are defined to be strictly periodic, deadline misses or not, do not have any impact on the job arrivals. Therefore, we do not require any assumption of the type of relative deadlines. However, we implemented *LazyTick* in FreeRTOS which enforces all jobs of one periodic task to share one task control block (TCB). Therefore, before a prior job of task τ_i finishes its execution, the subsequent job cannot be properly released due to the shared TCB. This would result in a management issue when the relative deadline is greater than the period. As a result, the implementation of *LazyTick* in FreeRTOS assumes constrained deadlines, but the method itself does not.

as the least-common-multiple (LCM) of $\mathcal{T}$. By releasing the first job of all tasks synchronously at time zero, their new jobs can only be dispatched at integer multiples of the periods of the tasks. Hence, the release time of the k -th job of a task τ_i is given by $k \cdot T_i$. The set of jobs that a task τ_i releases is denoted as $\mathbb{J}_i$, while the set of all jobs is denoted as $\mathbb{J}$. To determine which job $j \in \mathbb{J}$ the system executes at a given time $t \in \mathbb{R}$, we define a function $\sigma : \mathbb{R} \rightarrow \mathbb{J} \cup \{\perp\}$. If $\sigma(t) = j$, the system executes job j and if $\sigma(t) = \perp$, the system idles. In practice, for example in FreeRTOS, the system runs a special idle task at that time.

4 Sources of Overhead

In the following, we examine the two primary overheads incurred by the job release process, i.e., the tick interrupt overhead and the waiting set management overhead. These considerations motivate the development of *LazyTick*.

4.1 Tick Interrupt Overhead

As briefly introduced in the introduction, key housekeeping operations in an RTOS happen during a tick interrupt, invoked by a hardware timer. Every tick interrupt comprises three housekeeping operations, namely 1) incrementing the tick counter, 2) determining which task should release a job at the current tick and organizing the appropriate data structures, and 3) determining which job should be executed after the interrupt has finished. The resulting overhead can be decoupled into a *static* and *dynamic* part. The invocation of the interrupt itself and incrementing the tick counter are overheads with a constant cost which is not dependent on the current state of the system. However, the runtime for determining and releasing new jobs can differ vastly depending on how the waiting set is organized and how many tasks are currently waiting or having jobs in the ready queue. Lastly, performing a scheduling decision can be impacted by the means in which the global ready queue is organized. In the following, we first discuss the source of the *static* overhead and continue with the *dynamic* overhead in Section 4.2.

The *static* overhead is mainly determined by the number of invoked tick interrupts. We classify tick interrupts into *required* and *not-required* interrupts. A *required* interrupt is defined as a tick interrupt where a task releases a new job. After such interrupts, the currently executed job may change. A *not-required* interrupt only increments the tick counter and releases no additional jobs and therefore does not change the job that is currently executed. Since an RTOS needs to exactly meet the time when any task should release a new job, *required* interrupts are necessary to ensure a correct execution of a task set. To reduce interrupt-related overheads, it is essential to minimize the number of *not-required* interrupts, as each interrupt stalls the execution of any ongoing task.

Implementations that use a one-shot timer—triggering an interrupt only when a new job is released—inherently avoid *not-required* interrupts. However, reconfiguring the timer interval introduces additional management overheads. First, the timer itself needs to be reconfigured, which is strongly dependent on the underlying hardware capabilities. The reconfiguration can have a high overhead or even be outright impossible if the timer can only be configured offline. Additionally, the realization when the timer interval needs to be updated poses overheads. If it is only updated during a tick interrupt, next-to-be-released job is not necessarily contained in the waiting set. In that case, all tasks must be inspected to determine when the next job release will occur. Alternatively, the alarm interval can be updated any time a job finishes its execution. The task is then inserted into the waiting set, and its period can be adopted as the timer interval, if the current one-shot timer interval is greater than the period of the task.

For systems with periodic tick interrupts, the timer interval is fixed and has to be set as the greatest common divider (GCD) of all task periods in order to not miss any job releases. Consequently, depending on the task set, periodic timer interrupts may lead to *not-required* interrupts, but it

requires no dynamic management of timer intervals. From an analysis point of view, the overheads generated by periodic timers are predictable. We can determine the number of invoked interrupts from a single timer with a period P over a given time frame, e.g., the hyperperiod H , by $\eta = \frac{H}{P}$. This can be generalized for a system with multiple timers as:

Overhead Source 1 (Number of Interrupts). *Given a task set $\mathbb{T}$ with task periods $\mathcal{T}$ and a set of periodic timers $\mathbb{P}$, the number of tick interrupts during one hyperperiod $H = \text{lcm}(\mathcal{T})$ is $\eta = \sum_{p_j \in \mathbb{P}} \frac{H}{p_j}$.*

To make use of multiple timers, *LazyTick* partitions the task set such that each subset is assigned to a separate periodic timer. This partitioning allows each timer to operate with a longer interrupt interval, thereby reducing the overhead of Overhead Source 1. Since we employ periodic timers, even though there are multiple tick interrupt sources, *LazyTick* maintains the predictability of strictly periodic interrupts, as each timer still operates on a fixed interval.

4.2 Waiting Set Management Overhead

In RTOSes, commonly there are two data structures relevant to the job release process, namely the *waiting set* and the *ready queue*. Consider a recurrent task τ , which releases an infinite number of jobs. At tick t , depending on whether task τ has a job that has been released but not yet finished, either the *ready queue* holds a job of the task, or the task itself is placed in the *waiting set*. If there is a job of the task τ released to the system (i.e., ready to be executed), then the released job is in the *ready queue* at tick t . Otherwise, at time t , there is no job of the task τ released to the system and τ is in the *waiting set*. The status of the *ready queue* at tick t can be described by Equation (1).

$$\mathcal{R}_t = \{j_k \in \mathbb{J}_i \mid \tau_i \in \mathbb{T}, t \geq k \cdot T_i, \sigma(t) \neq j_k, j_k \text{ has not finished}\} \quad (1)$$

The *ready queue* contains all jobs from tasks that are ready to be executed at tick t , besides the currently executing job. The *waiting set* at tick t can be described formally by Equation (2).

$$\mathcal{W}_t = \{\tau_i \in \mathbb{T} \mid t < k \cdot T_i, j_{k-1} \in \mathbb{J}_i \text{ has finished}\} \quad (2)$$

It contains any task whose last job has finished and is not yet ready to release a new job.

In this work, we restrict our scope to the management of the waiting set. The management of the waiting set strongly impacts the *dynamic* overhead of the tick interrupt and the overhead that has to be paid every time a job finishes its execution. During a tick interrupt, in order to efficiently determine which tasks in $\mathcal{W}_t$ should release a new job, keeping $\mathcal{W}_t$ sorted is common practice in RTOSes. This however, influences the overhead of delaying a task τ_i whenever one of its jobs has finished its execution. If the waiting set should be kept sorted, the insertion of τ_i has to be performed in a sorted manner, resulting in an overhead that is dependent on the current state of $\mathcal{W}_t$. As a result, if $\mathcal{W}_t$ is kept sorted, the overhead at the end of any job execution can differ vastly, making the system-related overhead less predictable. In addition, this variability introduces further pessimism into the WCET of every task. Alternatively, if $\mathcal{W}_t$ is kept unordered, τ_i can just be appended, resulting in a constant overhead. However, during a tick interrupt, $\mathcal{W}_t$ has to be inspected completely. Naturally, in both realizations, releasing tasks from $\mathcal{W}_t$ results in a runtime of $O(|\mathbb{T}|)$ in the worst-case if every task releases a job. The overhead induced by the waiting set management is summarized in the following:

Overhead Source 2 (Managing the Waiting Set). *The management of the waiting set consists of two aspects: 1) during a tick interrupt, the minimal elements according to the next job release time have to be retrieved from the waiting set. 2) Whenever a job has finished its execution, the originating task has to be inserted into the waiting set at the correctly sorted position.*

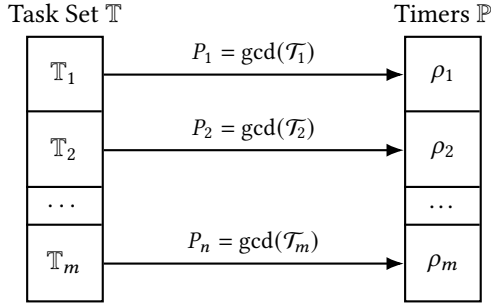


Fig. 1. Mapping of m partitions of a task set $\mathbb{T}$ to m timers.

In general, Overhead Source 2 can be minimized by precomputing which task will release a job for every time unit within one hyperperiod, enabling constant-time operations per task. However, the memory footprint of the resulting table makes this approach unrealistic for most system configurations. *LazyTick* addresses this challenge by exploiting the properties of harmonic task sets and the availability of multiple hardware timers to achieve a waiting set with minimal overheads.

5 Partitioning the Task Set

Towards decreasing the impact of Overhead Source 1, the periods of the timers should be maximized. Since an interrupt must always occur at exactly the release time of every job, it is not possible to arbitrarily increase the timer periods, but the tick interval always needs to be an integer divisor of all task periods. To minimize the number of tick interrupts, the timer period should therefore adopt the GCD of the task periods. For systems with a single periodic timer, this often results in a timer that needs to interrupt at every tick. However, in systems equipped with multiple timers, where each can generate interrupts at a different interval, we can exploit them by partitioning the task set and assigning each timer a separate subset, allowing for increased timer periods.

Here we provide an example to illustrate our idea. Suppose the task set contains the following periods $\mathcal{T} = \{3, 9, 7, 14\}$. A system with only one timer has to trigger a tick interrupt every time unit to not miss the release time of any task since 3 and 7 are co-prime. If the system contains two timers ρ_1, ρ_2 , the task set can be partitioned such that tasks with periods $\mathcal{T}_1 = \{3, 9\}$ are handled by timer ρ_1 , and the tasks with periods $\mathcal{T}_2 = \{7, 14\}$ are handled by timer ρ_2 . Then, the period P_1 of timer ρ_1 can be configured to 3 time units and the period P_2 of ρ_2 can be set to 7 time units. Within one hyperperiod, a system using a single timer experiences 126 tick interrupts, whereas employing two timers reduces the number to 60. As a result, if there are sufficiently many timers for a given task set, the number of tick interrupts can be effectively decreased, compared to a system that contains only one periodic timer. Please note that, when the number of available timers is insufficient to fully partition the task set, the timer interval must degrade to the GCD of the task periods in its assigned subset—potentially as low as one—to ensure correct job releases. As a result, the benefits of reducing interrupt frequency diminish with fewer timers.

5.1 Mapping Tasks to Timers

For a system with multiple periodic timers $\mathbb{P}$, we propose a strategy to partition a given task set $\mathbb{T}$ and assign each partition to a separate periodic timer. In order to minimize the number of generated tick interrupts while ensuring a correct job release, each timer then adopts the GCD of the task periods of its assigned partition. This concept is illustrated in Figure 1.

Suppose that the system contains a set of m hardware timers $\mathbb{P} = \{\rho_1, \dots, \rho_m\}$ that cause periodic interrupts at $P_1, \dots, P_m$ time units. We then partition the task set $\mathbb{T}$ into $\ell \leq m$ partitions $\Gamma = \{\mathbb{T}_1, \dots, \mathbb{T}_\ell\}$ and assign the tasks of each partition to a separate timer. We do not necessarily use all m available timers, as it is not always beneficial to configure all available timers. For example, for a task set with periods $\mathcal{T} = \{3, 9, 7, 14\}$ and a system with three available hardware timers ρ_1, ρ_2, ρ_3 , it is sufficient to use two timers ρ_1, ρ_2 that generate interrupts every 3 and 7 time units respectively. Employing a third timer ρ_3 does not cover any additional tasks beyond those already handled by ρ_1 and ρ_2 , and therefore does not further reduce the total number of generated interrupts.

5.2 Mapping Strategy

We define the problem of finding a mapping of tasks to timers that minimizes the number of tick interrupts while ensuring correct job releases as an optimization problem. For the mapping, we need to ensure that two properties are satisfied. First, every task $\tau_i \in \mathbb{T}$ needs to be assigned to exactly one timer ρ_j . Since all tasks are released synchronously at time zero, all tasks that share the same period can be assigned to the same timer without affecting the number of tick interrupts or the correctness of the job release. The second property that needs to be satisfied is that for a set of tasks $\mathbb{T}_j \in \Gamma$ that is assigned to a timer ρ_j , the timer interrupt interval P_j has to be a divisor of all task periods $\mathcal{T}_j = \{T_i \mid \tau_i \in \mathbb{T}_j\}$. Otherwise, the release time of a job will be missed. To state the objective of the optimization problem as a minimization of the number of tick interrupts, it suffices to consider the number of interrupts during one hyperperiod. The objective and the properties are defined formally in the following as constraints of an MIQCP:

Let $|\mathcal{T}| = n$ and $|\mathbb{P}| = m$, then towards ensuring the property that every task needs to be assigned to exactly one timer, we define a boolean variable $\omega_{i,j}$ for every task with period $T_i \in \mathcal{T}$ and timer $\rho_j \in \mathbb{P}$. If $\omega_{i,j} = 1$, then tasks with the period T_i are assigned to timer ρ_j . To ensure that every task is assigned to exactly one timer, Equation (3) needs to be satisfied.

$$\sum_{j=1}^m \omega_{i,j} = 1 \quad \text{for } i = 1, \dots, n \quad (3)$$

To define the second property formally, we introduce an integer variable P_j for every timer ρ_j that represents the timer's period. Any timer period is at least set to one time unit and at most set to the maximum task period $\max(\mathcal{T})$. For a task τ_i that is assigned to timer ρ_j , its period P_j needs to be an integer divisor of T_i , i.e., a natural number $d_i \in \mathbb{N}$ needs to exist, such that $d_i \cdot P_j = T_i$ holds. Together with the mapping variables $\omega_{i,j}$, the property is defined in Equation (4).

$$d_i \cdot \sum_{j=1}^m P_j \cdot \omega_{i,j} = T_i \quad \text{for } i = 1, \dots, n \quad (4)$$

As for a task τ_i , only exactly one $\omega_{i,j}$ with $j \in [1, m]$ can be set to one, $\sum_{j=1}^m P_j \cdot \omega_{i,j}$ returns the period of the timer that task τ_i is assigned to.

The objective of minimizing the number of tick interrupts can be formulated as a minimization of the sum of the number of interrupts caused by every configured timer. For this, we introduce an integer variable η_j for every timer $\rho_j \in \mathbb{P}$. Every configured timer generates at least $\frac{H}{\max(\mathcal{T})}$ interrupts if the period of the timer is set to the longest task period. At most H interrupts are generated by any timer, if the timer period is set to one. Then, the number of interrupts caused by ρ_j during one hyperperiod is defined as Equation (5).

$$\eta_j = \frac{H}{P_j} \quad \text{for } j = 1, \dots, m \quad (5)$$

Since the hyperperiod H is the LCM of all task periods $\mathcal{T}$ and every timer period is an integer divisor of some task periods $\mathcal{T}' \subseteq \mathcal{T}$, the number of tick interrupts generated by a timer during one hyperperiod is always an integer.

As it may be beneficial to not configure all available timers, we introduce a binary variable u_j that denotes whether timer ρ_j should be used. If $u_j = 1$, then the timer ρ_j is configured. Whether a timer should be used, can be obtained by evaluating the mapping variables $\omega_{i,j}$. If for a timer ρ_j , any variable $\omega_{i,j}$ with $i \in [1, n]$ is one, u_j is set to one. Equation (6) defines this generally.

$$u_j = \bigvee_{i=1}^n \omega_{i,j} \quad \text{for } j = 1, \dots, m \quad (6)$$

Together with Equations (5) and (6), we can state the objective as Equation (7).

$$\min \sum_{j=1}^m \eta_j \cdot u_j \quad (7)$$

Towards solving the optimization problem, we can utilize Equations (3)–(7) to form the MIQCP. To this end, we rewrite Equation (4) as a quadratic function by introducing an auxiliary variable $a_{i,j}$ for every combination of $P_j \cdot \omega_{i,j}$. Then, the MIQCP can be formed with the following two equations in place of Equation (4).

$$a_{i,j} = P_j \cdot \omega_{i,j} \quad \text{for } i \in [1, n], j \in [1, m] \quad (8)$$

$$d_i \cdot a_{i,j} = T_i \quad \text{for } i \in [1, n], j \in [1, m] \quad (9)$$

Please note that the resulting MIQCP is non-convex. Since all variables are bounded, it is still possible to always compute a global optimum. After computing an optimal solution for the MIQCP, the mapping of tasks to timers can be extracted by evaluating the $\omega_{i,j}$ variables. The configuration of the timers is given by the P_j and u_j variables. An optimal solution can, for example, be computed with a solver like Gurobi [10].

Additionally, since all variables are bounded, the objective function and all constraints can be linearized. This means, it is possible to phrase the optimization problem as an integer-linear program (ILP). However, the linearization of the objective function and the constraints introduces a large number of additional variables, significantly expanding the solution space. As a result, finding an optimal solution for the ILP may become more challenging in practice.

6 LazyTick Job Release

Usually, in an RTOS that makes use of only one timer, there is a global system tick counter and one centralized waiting set that handles all tasks. In the following, we first extend this system architecture to multiple timers by introducing local tick counters and waiting sets per timer. With the mapping of tasks to multiple timers derived in Section 5, we then explain how to exploit the smaller task sets and longer timer periods for both harmonic and non-harmonic task sets to reduce the impact of Overhead Source 2.

6.1 Multiple Timer Tick Sources

When partitioning the task set and mapping subsets to multiple timers, *LazyTick* replaces the centralized waiting set, and the global system tick by local structures associated with each timer. The global waiting set is replaced by per-timer `timerSets`, each of which is managed independently by its corresponding timer. Whenever a job of a task has finished its execution, the task is inserted into the `timerSet` of its associated timer. Hence, the interrupt service routine (ISR) of every timer inspects only the associated `timerSet`. As the task set is partitioned and each associated timer

can invoke a tick interrupt at different periods, the time management in the RTOS is no longer centralized. To this end, each timer keeps its own tick counter, on which the associated tasks base their job release times. In order to keep the local times of each timer synchronous, all timers base their internal counters on a shared hardware clock source.

6.2 Harmonic Task Sets

Given a partition Γ of the task set $\mathbb{T}$, the properties of a task set assigned to a specific timer can be exploited. A task set $\mathbb{T}_j \in \Gamma$ is called *harmonic* if the period T_2 is an integer multiple of T_1 if $T_2 \geq T_1$ for any two tasks $\tau_1, \tau_2 \in \mathbb{T}_j$. In real-time applications, many tasks exhibit (semi-) harmonic periods [4, 9, 14, 16], and many techniques exploit this property to improve the system design [15, 19, 28].

Algorithm 1: *LazyTick* ISR for harmonic task sets

Input: Timer number j
Require: $P_j > 0$, timerSet_j sorted ascending by task periods

```

1 tick( $\rho_j$ ) +=  $P_j$ 
2 for  $i \leftarrow 1$  to  $|\mathbb{T}_j|$  do
3   if tick( $\rho_j$ ) mod  $T_i \neq 0$  then
4     break
5   end
6    $\tau_i \leftarrow \text{timerSet}_j[i]$ 
7   add  $\tau_i$  to ready queue
8 end
```

The waiting set is constructed as follows: At every index of the array, a descriptor of the task, e.g., the TCB, is stored. The TCBs in the array are sorted after the periods of the tasks in ascending order, in which ties are broken arbitrarily. Formally, for a timer ρ_j and the associated harmonic task set $\mathbb{T}_j = \{\tau_1, \dots, \tau_k\} \subseteq \mathbb{T}$, the timerSet_j is an array of k elements that stores the tasks $\tau_1, \dots, \tau_k$ sorted ascending by their periods $T_1, \dots, T_k$. This results in a size of $|\mathbb{T}_j|$ for the waiting set of timer ρ_j . Since all task periods are harmonic, ρ_j can be configured to generate tick interrupts with the same period as the smallest task period in $\mathbb{T}_j$, i.e., $P_j = \min(\mathcal{T}_j)$. Thus, *LazyTick* introduces no *not-required* tick interrupts, minimizing Overhead Source 1.

Towards minimizing Overhead Source 2, since the index of the task in the array is pre-determined and fixed, the insertion of a task into the waiting set is an $O(1)$ operation. Moreover, during a tick interrupt, checking whether any task is ready to release a job can also be realized efficiently. Any time the tick interrupt occurs, the timerSet is inspected from the beginning until a TCB is encountered where the period is not an integer multiple of the local timer tick. All other inspected tasks release a new job. The ISR for harmonic partitions is stated in Algorithm 1.

6.3 Non-harmonic Task Sets

For a non-harmonic task set, the order in which jobs are released is not fixed. For example, suppose a timer ρ_j is assigned a task set $\mathbb{T}_j = \{\tau_1, \tau_2\}$ with periods $\mathcal{T}_j = \{4, 6\}$, then $P_j = \gcd(\mathcal{T}_j) = 2$. The job releases handled by the ISR for one hyperperiod is shown in Figure 2. It can be observed, that the order in which the tasks need to release jobs is $\{\tau_1\} \rightarrow \{\tau_2\} \rightarrow \{\tau_1\} \rightarrow \{\tau_1, \tau_2\}$.

6.3.1 Timing Wheel. We adopt the *timing wheel* proposed by Varghese and Lauck [27] with efficient adaptations as we consider periodic tasks. A timing wheel according to Scheme 4 in [27] is a fixed size circular buffer that enumerates a given time frame F . Each element represents a time point

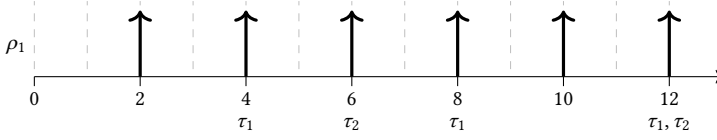


Fig. 2. Interrupts caused by one timer with period 2. An upward arrow indicates an interrupt. (Note that τ_1 and τ_2 release their first jobs not due to the interrupts but when the system starts.)

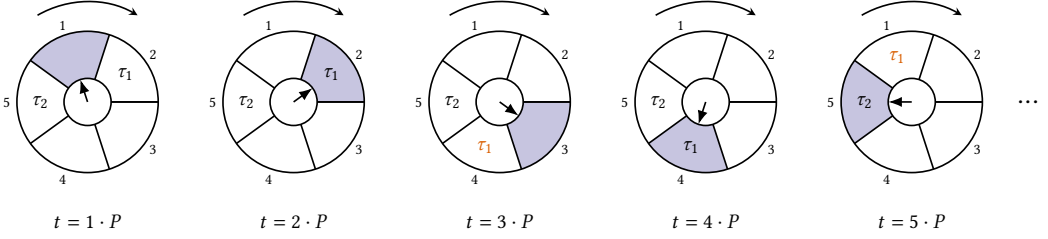


Fig. 3. *LazyTick* job release for non-harmonic task sets with $P = 4$, $T_1 = 8$, and $T_2 = 20$.

and contains a list of timers that expire at that time. The timing wheel advances one array element with each time unit. Therefore, the size of the circular buffer is given by the time frame F . Each time the wheel is advanced, the list at the current index is inspected. If any timers are contained in that list, expiry processing is performed on all contained timers. A new timer which expires at j time units after the current time i is inserted into the timing wheel at the position $i + j \bmod F$.

6.3.2 Timing Wheel for Periodic Tasks. Since we consider periodic tasks, we utilize Scheme 4 in [27] with the following adaptations:

- The granularity of the timer period is the GCD of the periods of the tasks in $\mathbb{T}_j$, instead of one time unit in [27].
- *LazyTick* only needs a simple circular buffer mechanism, instead of more complex hierarchical or hashing extensions in [27].
- We do not require any sorting, whilst sorting is still required in [27].

As the timer ρ_j for a task set $\mathbb{T}_j$ generates interrupts every $P_j = \gcd(\mathcal{T}_j)$ time units, we do not need to enumerate all time units for a given time frame. The timing wheel used in *LazyTick* advances by P_j time units at every tick interrupt. Therefore, a task τ_i is inserted at the position $p' = (p + \frac{T_i}{P_j}) \bmod |\text{timerSet}_j|$, if it was previously stored at position p . Initially, after τ_i has been released at the system start, it is inserted at position $p = \frac{T_i}{P_j}$. Since we store periodic tasks in the timing wheel, the maximal time frame that the wheel needs to handle is given by the maximal task period in $\mathcal{T}_j$. Therefore, the size of the timing wheel in *LazyTick* is given for a timer ρ_j by $|\text{timerSet}_j| = \max \left\{ \frac{T_i}{P_j} \mid T_i \in \mathcal{T}_j \right\}$. With this construction, the memory footprint of the timing wheel becomes manageable, making it unnecessary to use more complex structures. Furthermore, a task $\tau_i \in \mathbb{T}_j$ needs to be inserted at most $|\text{timerSet}_j|$ positions after its previous position, i.e., it will be inserted at the same position it was before. This means, there cannot be a case where it is required to sort the lists.

An example of a timing wheel for two periodic tasks τ_1, τ_2 with $T_1 = 8$ and $T_2 = 20$ that are assigned to a timer with period $P = 4$ is illustrated in Figure 3. Here, τ_1 is initially stored at position

2 and τ_2 at position 5. Every time a job of τ_1 has finished, τ_1 is advanced by two positions in the buffer. τ_2 always needs to be advanced by five positions, effectively staying at the same position.

The process of releasing jobs during a tick interrupt is described in Algorithm 2. First, every interrupt increments the local tick counter by the period of the timer. Then, the list at the current position p is inspected. If a task is in the list, we release a job and subsequently remove the TCB. As soon as the inspected list is empty, all necessary jobs have been released and the position of the timing wheel can be advanced by one, thereby ending the ISR.

Algorithm 2: *LazyTick* ISR for non-harmonic task sets

Input: Timer number j

Require: $P_j > 0$, current circular buffer position p

```

1 tick( $\rho_j$ ) +=  $P_j$ 
2 loop
3   if timerSet $_j[p]$  is empty then
4     | break
5   end
6    $\tau \leftarrow \text{head}(\text{timerSet}_j[p])$ 
7   remove  $\tau$  from timerSet $_j[p]$ 
8   add  $\tau$  to ready queue
9 end
10  $p \leftarrow (p + 1) \bmod |\text{timerSet}_j|$ 

```

Since it is always known which list needs to be inspected and all tasks that need to release a job are contained in that list, releasing jobs at each tick introduces little overhead. Also, our timing wheel never needs any sorting, making insertions always an $O(1)$ operation. In summary, *LazyTick* reduces the impact of Overhead Source 2 for non-harmonic task sets.

7 Evaluation

This section presents the evaluation results of *LazyTick* implemented on FreeRTOS, against two alternatives: the default FreeRTOS timer implementation and a one-shot timer variant. Following the finding of Hagens and Chen that more complex structures increase the processing overhead [11], we retain the default data structures in FreeRTOS. The doubly-linked list that acts as the waiting set in the FreeRTOS baseline and one-shot implementation is maintained in sorted order based on the next release time of the tasks.

To evaluate the effectiveness of *LazyTick*, we examine the impact on the overhead when inserting tasks into the waiting set and on the overhead created by tick interrupts. In particular, we focus on the effect on the variability in runtime of the overheads. A low runtime variability results in more predictable overheads for real-time systems. We also examine the combined overhead of Overhead Source 1 and Overhead Source 2, alongside the memory footprint of the data structures.

Sections 7.1, 7.2, and 7.3 present the evaluation setup, measurement methodology, and test configurations, respectively. The runtime of solving the MIQCP instances is presented in Section 7.4. We evaluate large (≥ 50 tasks) and small task sets (10–50 tasks) in Sections 7.5 and 7.6, respectively, whilst Section 7.7 presents the evaluation results of automotive benchmarks. To assess the impact of the overheads on the practical schedulability of task sets, we also perform experiments to execute task sets on an ESP32 in Section 7.8. Section 7.9 summarizes the evaluation results.

7.1 Evaluation Setup

To evaluate *LazyTick* on a real-world embedded device, *LazyTick* was implemented in the ESP32 port of FreeRTOS v10.5.1, included in the ESP-IDF development framework in version v5.3.1. The unicorn version of FreeRTOS was used.

All experiments were conducted on an ESP32-S3-DevKitC-1, equipped with an ESP32-S3 chip and 8 MB of externally connected PSRAM. The microcontroller contains four timers that can be independently configured and share a common clock source. We use the built-in GPTimer library to configure the timers and implement the ISRs. For each test, the CPU clock frequency was set to 240 MHz. In order to evaluate large task sets without experiencing deadline misses, one system tick was configured to be equivalent to 100 ms for the tests in Section 7.5. For the small synthetic task sets with 10–50 tasks in Section 7.6, one system tick was set to 1 ms. We set the compiler optimizations level to -O0 to reduce the impact of compiler-based optimizations on the measurements. The solutions to the MIQCP instances were computed on a server system equipped with two AMD EPYC 9654 CPUs and 768 GB of RAM.

7.2 Measurement Methodology

In all tests, the insertion overhead into the waiting set and the cumulative runtime of tick interrupts is recorded for 1000 ticks. For the large synthetic task sets in Section 7.5, one tick represents 100 ms and thus each test runs for 100 s. Since one system tick was configured to 1 ms for the tests in Section 7.6, these experiments run for 1 s each. To determine the overhead of tick interrupts, a probe point was added at the start and the end of any interrupt. As there are other sources of interrupts in FreeRTOS, e.g., a yield call is realized via an interrupt, the source of the interrupt was inspected. Since the ESP32-S3 is based on the Xtensa [6] architecture, it supports 32 interrupt sources. When an interrupt is allocated on the system, a number between 0 and 31 is assigned to it that represents the interrupt source. On Xtensa architectures, the INTERRUPT register can be read to check which interrupt is triggered. Every bit of the register represents a different interrupt source. When the i -th bit of the INTERRUPT register is equal to 1, the interrupt source with the assigned number i caused the interrupt. In order to distinguish the tick interrupts from any other source, the mapping of interrupt sources to bits of the INTERRUPT register was fixed. The INTERRUPT register was read at the start of any interrupt to filter out any non-tick interrupts. At both probe points, the current clock cycle was recorded by reading the CCOUNT register to minimize the overhead of the time measurement.

The time spent on each insertion into the waiting set was also recorded by reading the CCOUNT register. To determine the overhead for retrieving which tasks should release a job during each interrupt, the retrieval times were recorded in the same manner. It should be noted, that this process happens solely during tick interrupts. Therefore, the time spent in a tick interrupt always includes the job retrieval time.

7.3 Test Configurations

We performed tests for task sets with up to 500 tasks. As the internal memory of the ESP32-S3 is limited to 512 kB [26], the stack memory of each task is allocated on the external PSRAM. This allows us to evaluate larger task sets where the overhead of the waiting set is more impactful. However, we were restricted to 500 tasks, as FreeRTOS on the ESP32 does not allow allocating TCBs on external memory, making the internal SRAM the limiting factor.

We generated specific harmonic and non-harmonic task sets for one to four timers with 50 to 500 tasks. For every scenario a mapping was computed with the MIQCP presented in Section 5 by adopting Gurobi [10] to find a globally optimal solution. The task sets were constructed as follows:

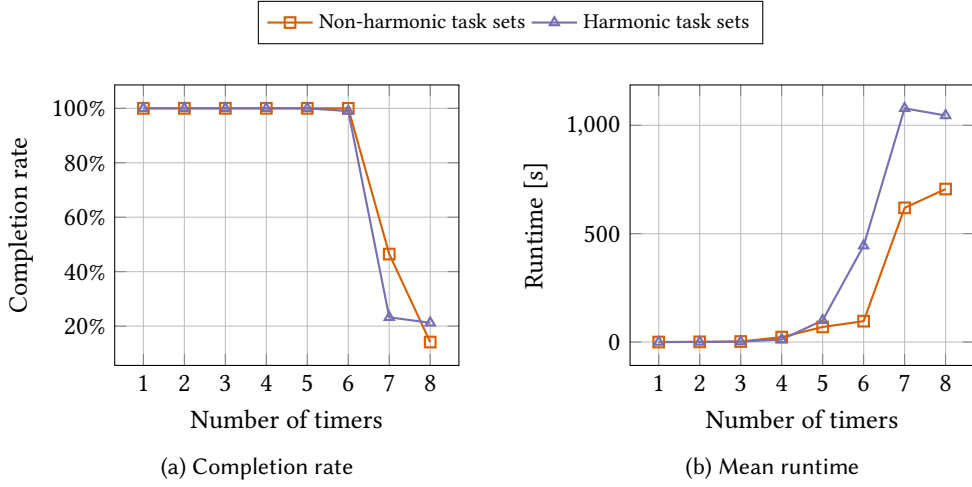


Fig. 4. Completion rate in one hour and mean runtime of the MIQCP.

- For harmonic task sets, based on the number of configured timers m , we selected m different base periods from the set $\mathcal{T}_b = \{5, 7, 11, 13\}$ and assigned the period P_j of timer ρ_j accordingly. For a task set with n tasks and m timers, we then create n tasks with periods randomly in the following manner. First a task was randomly assigned to a timer ρ_j . Then, the period of the task was set to $P_j \cdot 2^r$, where $r \in \{x \in \mathbb{N} \mid 0 \leq x \leq 4\}$ was randomly sampled from a uniform distribution to vary to task periods in each partition.
- For non-harmonic task sets, we constructed the task sets in a similar manner by setting the period of the task assigned to the timer ρ_j as $P_j \cdot r$, where $r \in \{x \in \mathbb{N} \mid 1 \leq x \leq 5\}$. As a result, the periods were shorter and more tasks were executed in the observed 1000 ticks. This design resulted in more insertions into the waiting set and more jobs to be retrieved in a tick interrupt, amplifying the overheads.

Since we focus on evaluating the overhead of the waiting set management and tick interrupts, the computational effort for each task is kept to a minimum by performing one integer addition.

Generally, the process from generating the task set, computing the mapping by solving the MIQCP instance, integrating it into FreeRTOS and finally flashing and running the experiment on the ESP is fully automated. The generation of the task set and solving the MIQCP instance with Gurobi are combined in one Python script. To integrate the mapping and timer configuration into FreeRTOS, the description of the task set is exported as an array in C and imported into a header file alongside the configuration of timer periods. The description of a task includes the timer that manages it. For each test run, the firmware of the ESP is rebuilt and flashed. The recorded data points are then transferred over UART.

7.4 MIQCP Runtime

We first show that the runtime measurements of solving the MIQCP instances. We solve MIQCP instances for task sets that are constructed as described in Section 7.3 for 5–500 tasks with a granularity of 5 tasks. We evaluate the convergence time for up to 8 timers. For 8 timers, the task set is constructed from the set of base periods $\mathcal{T}'_b = \{5, 7, 11, 13, 17, 19, 23, 29\}$. Note that our experiments on ESP (reported later in this section) can only make use of at most 4 timers due to

the hardware limitation. We impose a convergence time limit of one hour for each test scenario. The results are shown in Figure 4.

Figure 4a shows the percentage of MIQCP instances that were able to converge within the time limit of one hour on the y-axis and the number of timers on the x-axis. The plot shows that for up to 6 timers, an optimal solution was found for almost all scenarios within one hour. For larger numbers of timers, the completion rate reduces to around 20% at worst.

Figure 4b shows the time needed to solve the MIQCP instance for those scenarios where it converged within the time limit. The average runtime for a fixed number of timers is shown on the y-axis in seconds with the number of timers for each set of tests given on the x-axis. The plot shows that for up to 5 timers, the MIQCP instances converge within around a minute, while for larger number of timers the runtime increases significantly.

It should be noted that for the tested harmonic task sets, both the completion rate and mean runtime are worse than for generic task sets. This is due to the way the harmonic task sets are constructed. As described in Section 7.3, for the harmonic task set, the maximal period of a task assigned to timer ρ_j is given by $P_j \cdot 2^4$, while for the non-harmonic task sets, the maximal task period is given by $P_j \cdot 5$. As the number of variables in an MIQCP instance is affected by the maximal task period (cf. Equation (5)), the runtime is also affected thereby.

7.5 Large Synthetic Task Sets

In the following, we first show the measurements for the runtime variability of inserting tasks into the waiting set and the runtime variability of releasing jobs during a tick interrupt. Then, we show the combined overhead of tick interrupts and task insertions. Lastly, we compare the memory footprint of the baseline implementation using linked lists and the structures used by *LazyTick*.

7.5.1 Variability Measurements. We first report the results regarding *runtime variability*, measured in clock cycles by taking the difference between the longest and shortest observed insertion in a test scenario. The variability is measured for harmonic and non-harmonic task sets and compared between using *LazyTick*, the FreeRTOS default implementation, and a one-shot timer implementation using the same data structures as FreeRTOS. Figure 5 shows the runtime variability of inserting tasks into the waiting set every time a job has finished its execution by using up to four timers. We note that FreeRTOS and the one-shot implementation always uses one timer, while *LazyTick* uses all configured timers. The runtime variability is shown in clock cycles on the y-axis and the number of tasks is shown on the x-axis.

The plots show, that the observed runtime variability when using *LazyTick* stays steadily at around 2000 clock cycles, regardless of the number of tasks, configured timers or task set properties. In Figure 5a, the overhead for the baseline FreeRTOS implementation increases linearly with the number of configured tasks for the task set created for one timer. In Figure 5b, 5c, and 5d, each of the generated task sets contained more tasks with different periods by construction. Here, the runtime variability is less consistent and only roughly follows a linear trend. For the one-shot timer implementation, the plots show that the runtime variability converges to the default FreeRTOS implementation when there are enough tasks. Compared to *LazyTick*, the variability of both baseline approaches gets significantly higher and is strongly dependent on the number of tasks.

Figure 6 shows the runtime variability of determining which tasks should release jobs, and releasing them to the ready queue during a tick interrupt. We show measurements for harmonic and non-harmonic task sets comparing *LazyTick*, FreeRTOS and the one-shot timer implementation. The runtime variability was computed over the recorded runtimes of the job release operation in each tick interrupt and is shown on the y-axis. The number of tasks is shown on the x-axis.

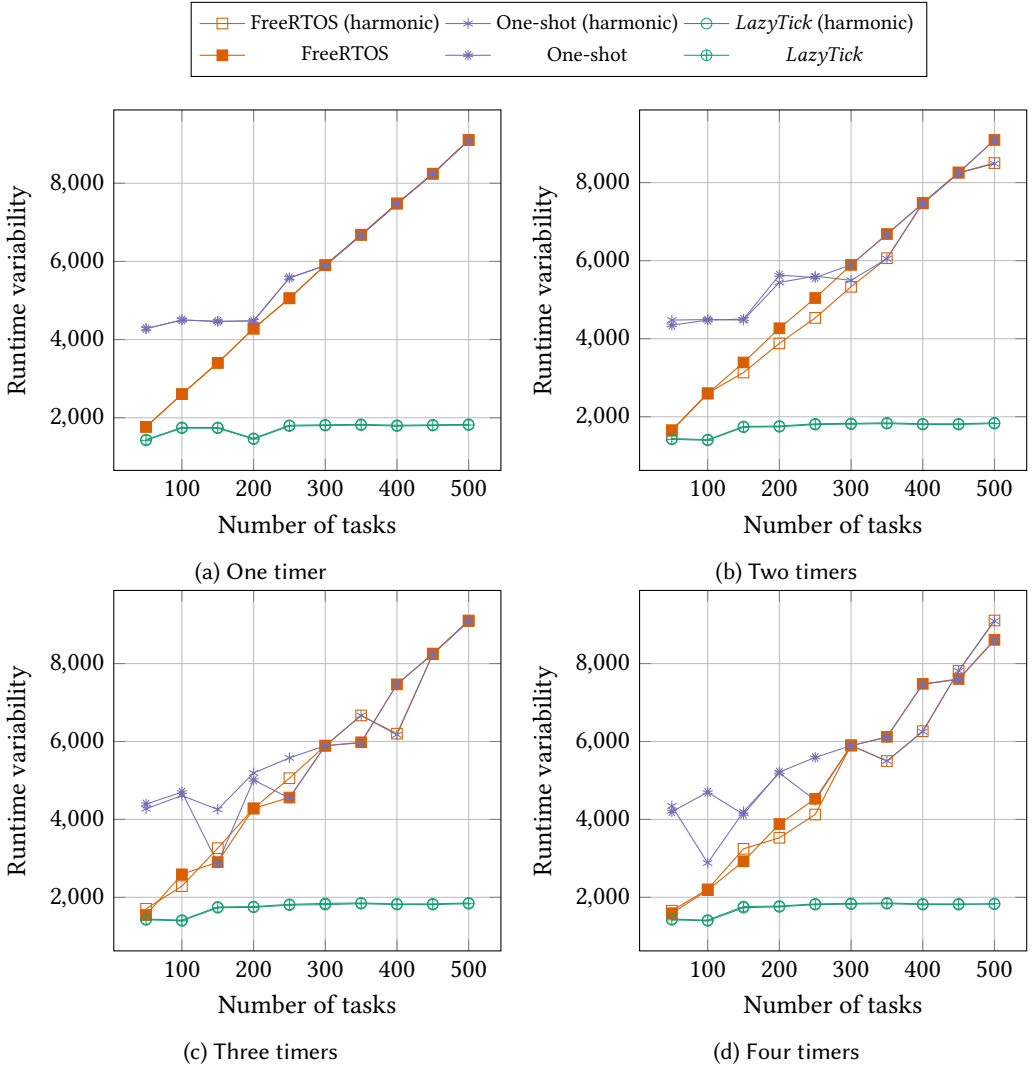


Fig. 5. Insertion runtime variability measurements (unit: clock cycles).

For all implementations in all test scenarios, the runtime variability increases linearly with respect to the number of tasks. In Figure 6a for a task set created with one timer, for harmonic task sets, *LazyTick* has lower runtime variability than the baseline implementations, whereas the one-shot implementation has the least runtime variability for non-harmonic task sets. In Figure 6b, 6c, and 6d, where multiple timers are used and the configured task sets contain more varied periods, *LazyTick* experiences the lowest runtime variability and a less steep increase than the other approaches. For harmonic task sets, the variability of *LazyTick* is noticeably lower than for non-harmonic task sets. For the other implementations, this observation is inverted, i.e., harmonic task sets experience a higher runtime variability. Still, the one-shot implementation is slightly more efficient than FreeRTOS in its default version. Notably, the runtime variability for three and four timers is significantly lower than for one and two timers. Since the task set is split over multiple waiting

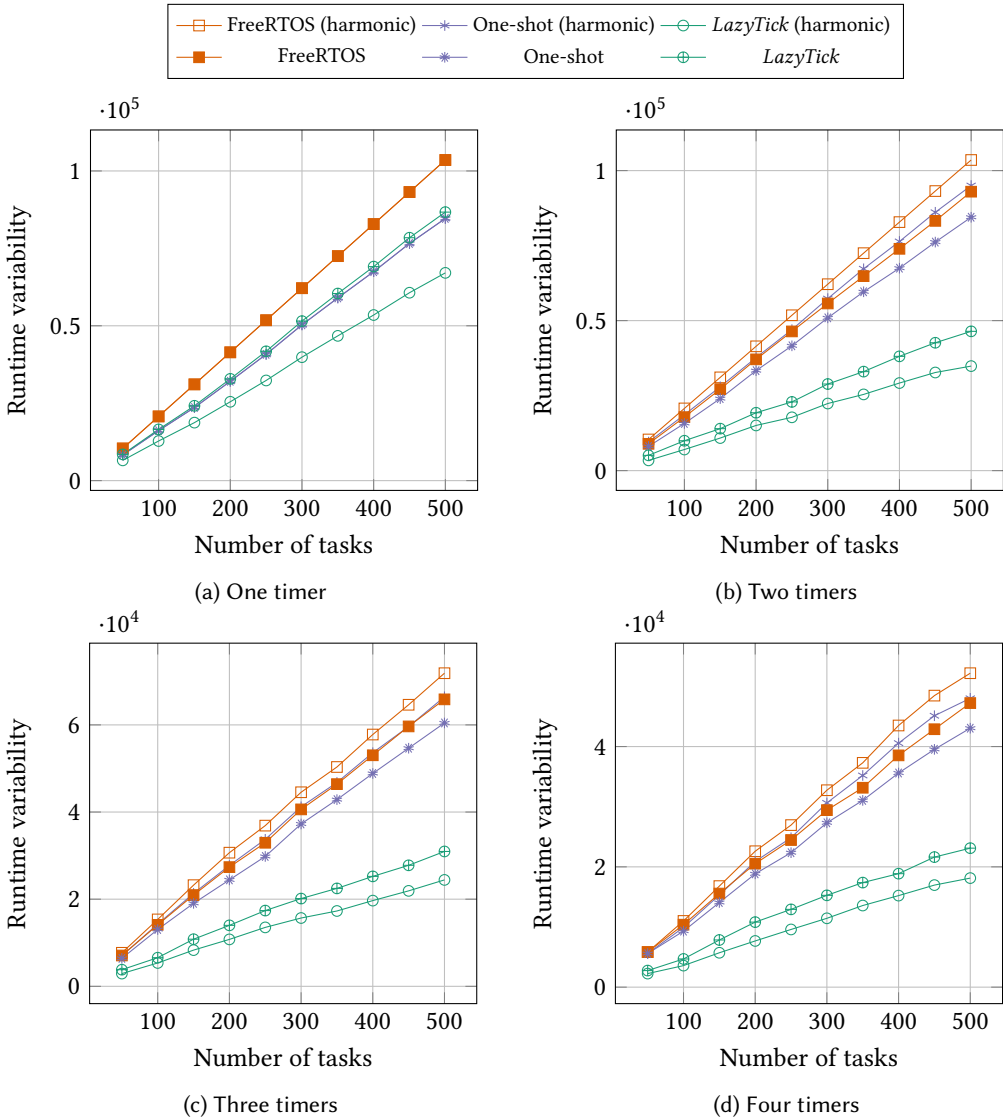


Fig. 6. Retrieval runtime variability measurements (unit: clock cycles).

sets when using *LazyTick*, fewer tasks need to be processed per timer. Thus, the runtime variability is noticeably reduced. Please note that, jobs are solely released during tick interrupts. As a result, a more consistent runtime also impacts the runtime variability of the tick interrupts themselves. In general, the lower and more steady runtime variability experienced by *LazyTick* when employing multiple timers leads to a more predictable behavior of job release operations in an RTOS.

It should be noted, while both the one-shot timer and default FreeRTOS implementations only use one timer, the results for different number of timers in Figure 5 and Figure 6 still differ as the task sets differ as well. Specifically, as described in Section 7.3, the task sets are built using different base periods depending on the number of timers. Thus, the task release patterns differ even though

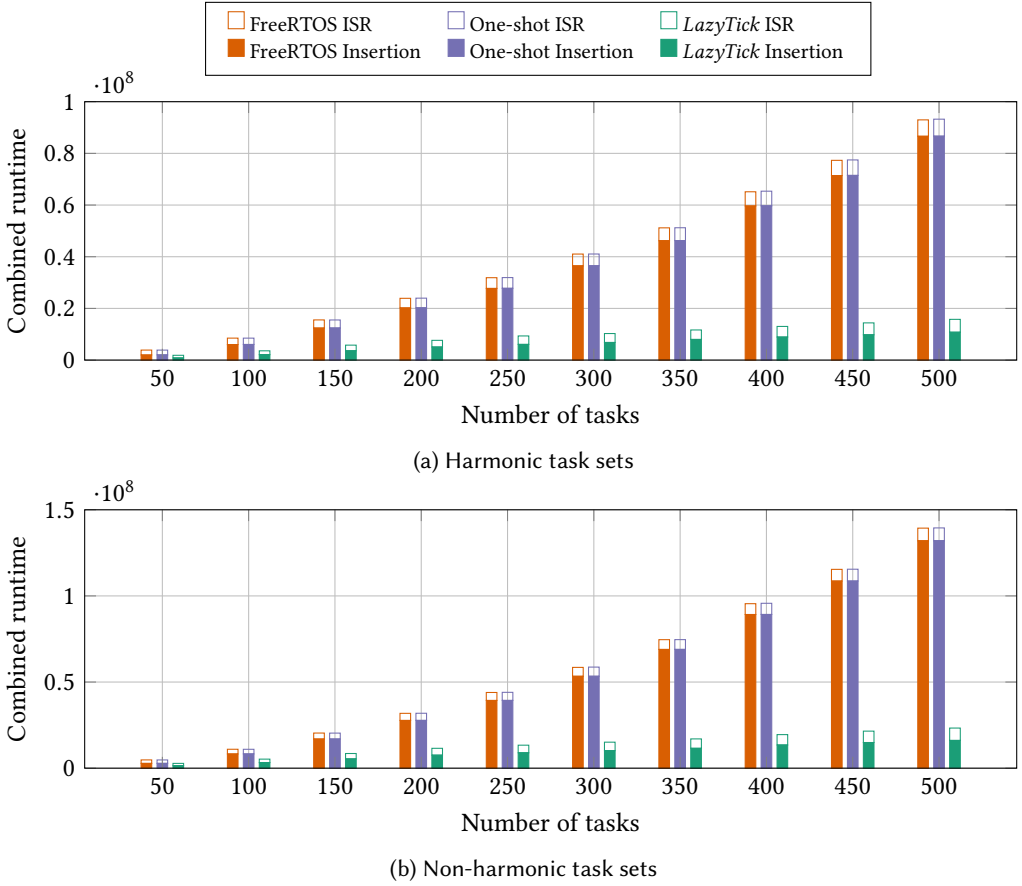


Fig. 7. Combined overhead for four timers (unit: clock cycles).

only one timer is used. As the runtime of inserting and retrieving tasks depends on the number of tasks in the data structures, the baseline methods experience different runtime variability.

7.5.2 Overhead Measurements. We have measured the combined overhead for task sets with up to four timers. Here, we report the results with four timers, as the others have similar trends. In Figure 7, the combined runtime overhead of the waiting set management and the tick interrupts is shown for harmonic task sets in Figure 7a and non-harmonic task sets in Figure 7b. On the y-axis, the plots show the combined runtime in clock cycles and the number of tasks on the x-axis. The bars show the overhead for inserting tasks as the bottom part and the tick interrupt runtime above. We summarize the combined overhead as the sum of all task insertions into the waiting set and all tick interrupts, i.e., it includes the job release process.

The plots show, that the overheads increase significantly for larger task sets, in particular when using the baseline approaches. From a task set with 50 tasks to a task set with 500 tasks, the combined overheads increase by $\approx 29\times$ for non-harmonic task sets and $\approx 24\times$ for harmonic task sets. On the other hand, when using *LazyTick*, the overhead increases by $\approx 8.4\times$ for non-harmonic task sets and $\approx 4.4\times$ for harmonic task sets. In all test cases, the observed overhead is also significantly lower when using *LazyTick*, with the gain in efficiency increasing with larger task sets. Also, the

Test scenario	100 tasks	200 tasks	300 tasks	400 tasks	500 tasks
FreeRTOS	34.02 kB	68.02 kB	102.02 kB	136.02 kB	170.02 kB
LazyTick (harmonic)	34.02 kB	68.02 kB	102.02 kB	136.02 kB	170.02 kB
LazyTick (non-harmonic)	34.56 kB	68.54 kB	102.58 kB	136.58 kB	170.56 kB
Table-driven (harmonic)	1.88 MB	3.76 MB	5.06 MB	6.53 MB	7.89 MB
Table-driven (non-harmonic)	7.98 MB	15.93 MB	21.96 MB	28.56 MB	34.70 MB

Table 1. Computed worst-case memory footprint with four timers.

plots show that, regarding the experienced overhead as a whole, the FreeRTOS and one-shot timer implementations incur similar overheads. Notably, the overhead for inserting tasks into the waiting set is vastly higher than the tick interrupt overhead for all approaches. In peak, the insertion overhead is up to $\approx 18.7\times$ higher than the overhead for tick interrupts.

7.5.3 Memory Footprint. We compare the memory footprint for the list-based implementation in FreeRTOS and the data structures used by *LazyTick*. Since the one-shot timer implementation solely changes the tick management, the memory footprint of the waiting set is the same as in the default FreeRTOS implementation. Also, for those two approaches the required memory for the waiting set stays the same, regardless of whether the task set is harmonic or non-harmonic. The worst-case memory footprint for task sets with four timers is shown in Table 1. In the worst case, the waiting set contains all tasks of the task set.

Generally, the memory footprints of these three approaches are very similar. For harmonic task sets, *LazyTick* uses the same amount of memory than FreeRTOS, as solely the overhead for the list data structure is replaced by pointers to the static arrays. For non-harmonic task sets, the memory footprint increases for *LazyTick* for the additional lists allocated in the circular buffer. Since the majority of the required memory stems from the tasks itself, the additional memory required is low.

We also compare to a table-driven approach by recording the actions needed in a table over one hyperperiod. The required memory to store the table for these task sets is vastly higher than that for the other three approaches presented above. In particular, the required memory increases even further for non-harmonic task sets, since the size is directly impacted by the length of the hyperperiod. For an embedded system, like the ESP32 used in our evaluation, a table-driven approach is therefore infeasible.

7.6 Small Synthetic Task Sets

In this section, we first show the measurements for runtime variability and subsequently the combined overhead of tick interrupts and task insertions for task sets consisting of 10–50 tasks.

7.6.1 Variability Measurements. The *runtime variability* is measured in the same manner as in Section 7.5. Figure 8 shows the runtime variability measurements for inserting tasks into the waiting set for task sets consisting of 10–50 tasks. The runtime variability is shown on the y-axis and given in clock cycles, while the number of tasks is shown on the x-axis. The plots show, that for smaller number of tasks the runtime variability of *LazyTick* also remains steady below 2000 cycles regardless of the number of tasks. Figure 8c and Figure 8d also shows that for the one-shot timer and default FreeRTOS implementation, the runtime variability is significantly impacted by the task set and ranges from below 2000 cycles to over 6000 cycles at worst. In general, their runtime variability is considerably more chaotic. Nevertheless, FreeRTOS experiences less runtime variability than *LazyTick* for some configurations when handling a very small number of tasks.

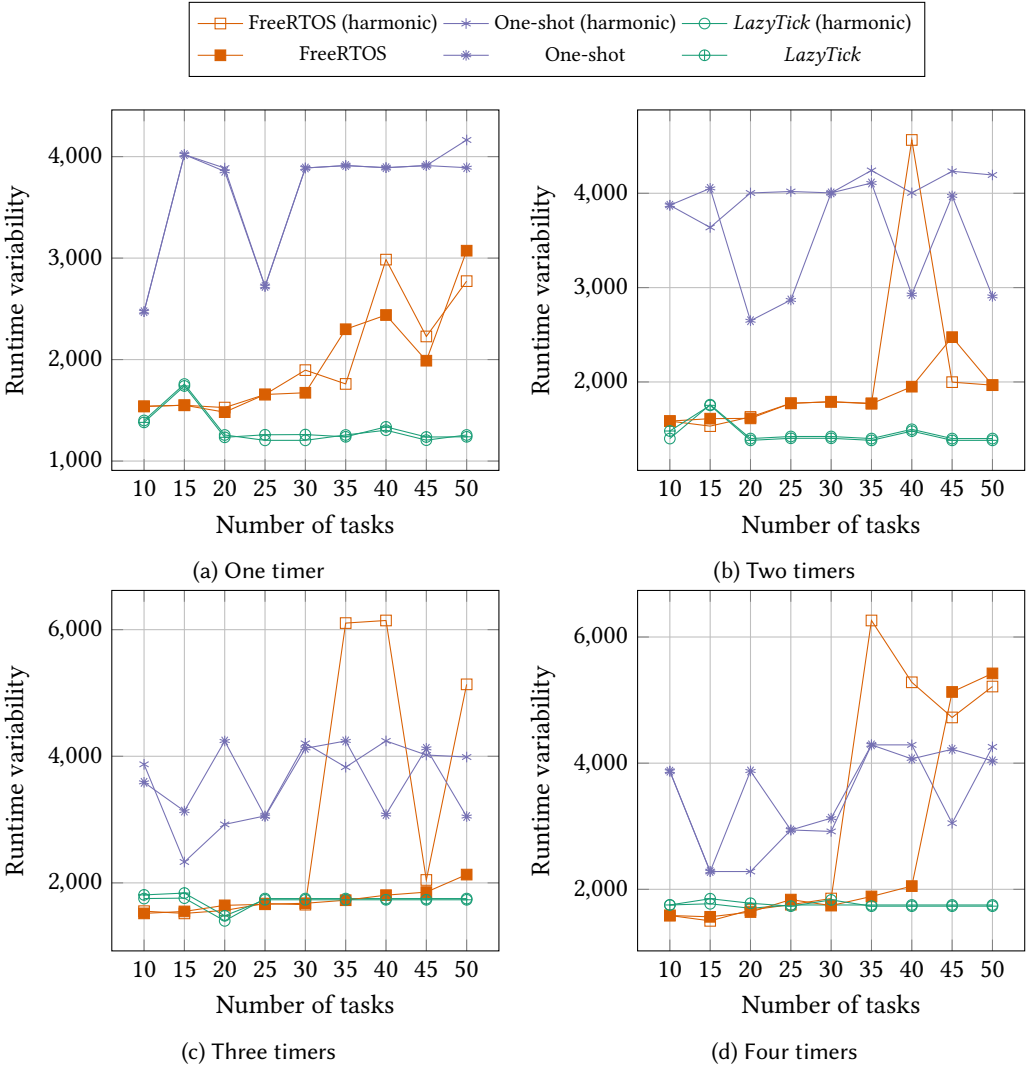


Fig. 8. Insertion runtime variability measurements for small task sets (unit: clock cycles).

Figure 9 shows the runtime variability measurements of determining which task should release a job and subsequently adding them to the ready queue during a tick interrupt. Here, for the smaller number of tasks, the plots for all scenarios follow the same trends as for the larger task sets. Especially when using multiple timers, *LazyTick* experiences a smaller runtime variability than the baseline solutions while also featuring a less steep increase for increasing numbers of tasks.

In general, it is important to note that these plots only show the runtime *variability*. A low variability does not imply a low absolute time overhead.

7.6.2 Overhead Measurements. For the smaller task sets, we have also measured the combined overhead for task sets with up to four timers. Since the results with fewer timers again show similar trends, we only report the results with four timers. In Figure 10, the combined runtime

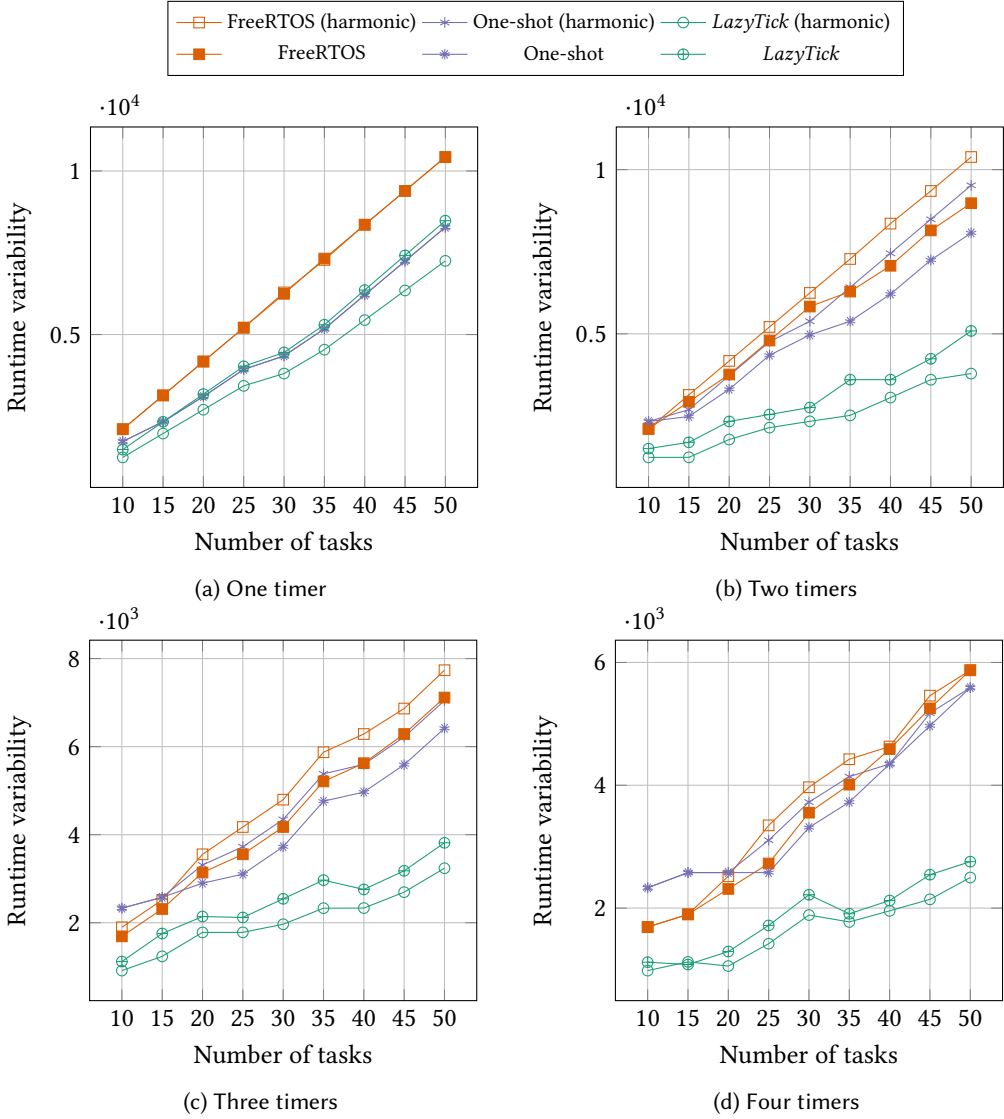


Fig. 9. Retrieval runtime variability measurements for smaller task sets (unit: clock cycles).

overhead of the waiting set management and the tick interrupts is shown for task sets with 10–50 tasks. Specifically, the results for harmonic task sets are shown in Figure 10a while the results for non-harmonic task sets are shown in Figure 10b. The plots show the combined runtime in clock cycles on the y-axis and the number of tasks on the x-axis. The filled out parts of the bars show the overhead for inserting tasks, while the parts above show the overhead caused by tick interrupts. As the most prominent difference to the measurements for larger task sets in Figure 7, the overhead for handling the tick interrupt is considerably more impactful for smaller numbers of tasks. Still, the trends shown in the plots are similar to those with larger task sets, with *LazyTick* causing only around half of the overhead compared to the baselines in all evaluated scenarios.

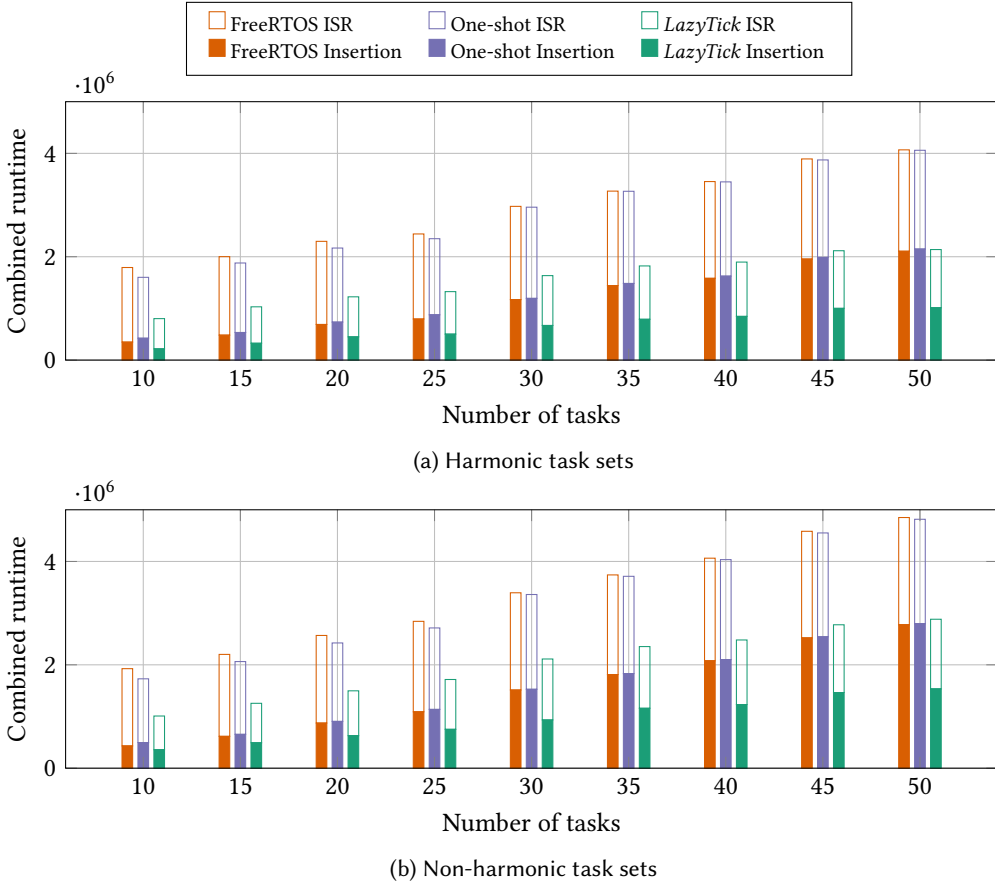


Fig. 10. Combined overhead for four timers and small task sets (unit: clock cycles).

7.7 Automotive Benchmarks

Besides the synthetic task sets evaluated in Section 7.5, we also evaluated *LazyTick* with task sets based on the automotive benchmarks [14], which have been widely adopted in the literature to assess design methodologies of real-time systems. We generated task sets with 50 to 500 tasks, including only periodic tasks. As the proposed task sets include tasks with period 1, for the full automotive benchmarks, a single periodic timer is sufficient to handle all job releases. To evaluate the effectiveness of *LazyTick* with multiple timers, we additionally generated task sets that exclude tasks with period 1 such that a partition can be found where two timers are employed. We evaluated the same metrics as for the synthetic benchmarks and observed similar results. Using *LazyTick* improves the runtime consistency of inserting tasks into the waiting set and reduces the overall overhead for task sets that cannot be partitioned well for the timer configuration of our evaluated system. Moreover, the cost for inserting tasks into the waiting set is the primary source of overhead.

7.8 Impact on Schedulability

We performed experiments with task sets that are at the border of being theoretically schedulable when neglecting overheads. For a task set with n tasks $\mathbb{T} = \{\tau_1, \dots, \tau_n\}$, we set the periods

	FreeRTOS	One-shot	LazyTick (harmonic)
# tasks without deadline misses	5 tasks	12 tasks	17 tasks

Table 2. Number of tasks that can be executed without deadline misses.

$T_1, \dots, T_{n-1}$ to n , and their execution times $C_1, \dots, C_{n-1}$ to 1 tick. The period of task τ_n is then set to $T_n = 2n$ and its execution time is set to $C_n = 2$. Moreover, we assigned each task a priority in descending order, i.e., τ_1 has the highest priority and τ_n has the lowest priority. Here we assume implicit deadlines, i.e., the relative deadline of a task is its period, and one system tick is defined as 10 ms. To enforce fixed execution time for a job, we utilized a busy-loop with a fixed number of additions. With the clock frequency of the ESP set to 240 MHz, a job execution took around 10 ms.

Since the task set is harmonic, it is schedulable with the utilization of 100% if there are no system overheads. In practice, the overhead of an RTOS is always non-zero resulting in τ_n always missing its deadline at tick $2n$. To evaluate whether the overhead reduced by *LazyTick* has an impact on the practical schedulability of a task set, we reduced the execution time C_n slightly by 1500 additions. We ran experiments with task sets comprised of 2–20 tasks since an increasing number of tasks leads to more overheads as seen in Figures 7 and 10. The results are shown in Table 2.

For up to 5 tasks, executing the task set experienced no deadline misses under all evaluated methods. The default FreeRTOS implementation encountered deadline misses, for task sets with more than 5 tasks. The one-shot timer implementation could execute up to 12 tasks without any misses. *LazyTick* experienced no deadline misses up to 17 tasks. As a result, a reduction of overhead does have a considerable effect.

7.9 Summary of Evaluation Results

Our evaluation shows that utilizing multiple timers to reduce the number of tick interrupts and adapting the waiting set to the properties of a given task set can result in significantly more consistent and reduced overheads for both small and large task sets. The summarized results can be found in Table 3. *LazyTick* reduces the runtime variability when inserting tasks into the waiting set up to $\approx 5\times$ in peak compared to both the FreeRTOS default implementation and the one-shot implementation in the tested scenarios. On average, *LazyTick* reduces the variability by up to $\approx 3\times$ compared to FreeRTOS and $\approx 3.4\times$ compared to the one-shot implementation, indicating that a reconfiguration of the timer is costly on an ESP32. *LazyTick* reduces the runtime variability of the job release process during tick interrupts by up to $\approx 3\times$ in peak and $\approx 2.5\times$ for harmonic task sets. For non-harmonic task sets, employing *LazyTick* results in a reduction of $\approx 2.3\times$ in peak and $\approx 1.7\times$ on average. Moreover, when regarding the combined overhead incurred by the waiting set realization and tick interrupts, employing *LazyTick* reduces the overhead by up to $\approx 6\times$ in peak and $\approx 3.6\times$ on average. Our evaluation shows, that especially the realization of the waiting set strongly impacts the combined overhead, as the insertion overhead was observed to be up to $\approx 18.7\times$ higher in peak. Our experiments also show, that reducing the overhead affects the size of the task sets that can be executed without experiencing deadline misses. Here, *LazyTick* can execute task sets with more than $3\times$ as many tasks as the baseline FreeRTOS approach.

In summary, exploiting multiple timers and adapting the waiting set data structure to a given task set raises the system efficiency notably. Additionally, adapting the job release process to a given task set also makes overheads incurred by housekeeping operations more consistent.

	Runtime Variability (insert)		Runtime Variability (retrieval)		Combined Overhead	
	peak	mean	peak	mean	peak	mean
FreeRTOS (harmonic)	4.99×	2.92×	3.07×	2.54×	6.07×	3.68×
One-shot (harmonic)	4.98×	3.38×	2.79×	2.29×	6.07×	3.67×
FreeRTOS (non-harmonic)	5.02×	3.01×	2.22×	1.79×	6.14×	3.63×
One-shot (non-harmonic)	5.01×	3.47×	2.38×	1.60×	6.14×	3.62×

Table 3. Reduction of runtime variability and overhead when using *LazyTick*.

8 Conclusion

In this work, we present *LazyTick*, an efficient realization for reducing both the number of redundant tick interrupts and the dynamic overhead of managing the waiting set. In particular, two job release procedures with constant-time operations per task are introduced for harmonic and non-harmonic task sets, respectively. To the best of our knowledge, this work is the first to tailor the management of job release based on the properties of task sets, while making use of available hardware timers.

It is worth noting that, our work not only improves RTOS efficiency, but also has the potential to reduce pessimism in worst-case timing analysis by minimizing overhead-related variability. We also note that, when the available number of hardware timers is not sufficient, the effectiveness of *LazyTick* might be limited since the timer intervals may degrade to low values (e.g., a GCD of 1).

In future work, we plan to generalize *LazyTick*'s applicability to multicore and distributed embedded platforms, where decentralized timer resources can be orchestrated. Moreover, the co-optimization of WCET and management overhead, where timer allocation and task partitioning are jointly considered to balance predictability and efficiency, is also a promising direction to explore.

Furthermore, in order to account for tasks with arbitrary phases (i.e., the first job of a periodic task is not released at time 0), we plan to explore solutions consisting of smartly assigning timers per each pair of unique task phase and period. For handling sporadic tasks, one future concept to be explored is to allow single jobs to be assigned to different existing periodic timers in the system. Analyzing the impact of that solution will be the key effort.

Another aspect of future work is tackling the complexity of the MIQCP. Heuristic algorithms as well as more lightweight optimization schemes, e.g., linear approximations, can be explored.

References

- [1] Benny Akesson, Mitra Nasri, Geoffrey Nelissen, Sebastian Altmeyer, and Robert I. Davis. 2022. A comprehensive survey of industry practice in real-time systems. *Real Time Syst.* 58, 3 (2022), 358–398. doi:10.1007/S11241-021-09376-1
- [2] Björn B. Brandenburg. 2011. *Scheduling and locking in multiprocessor real-time operating systems*. Ph.D. Dissertation. University of North Carolina, Chapel Hill, USA. doi:10.17615/X1ZQ-V169
- [3] Björn B. Brandenburg and James H. Anderson. 2009. On the Implementation of Global Real-Time Schedulers. In *Proceedings of the 30th IEEE Real-Time Systems Symposium, RTSS 2009, Washington, DC, USA, 1-4 December 2009*, Theodore P. Baker (Ed.). IEEE Computer Society, 214–224. doi:10.1109/RTSS.2009.23
- [4] José V. Busquets-Mataix, Juan José Serrano, Rafael Ors, Pedro J. Gil, and Andy J. Wellings. 1996. Using harmonic task-sets to increase the schedulable utilization of cache-based preemptive real-time systems. In *Third International Workshop on Real-Time Computing Systems Application (RTCSA '96), October 30 - November 01, 1996, Seoul, Korea*. IEEE Computer Society, 195–202. doi:10.1109/RTCSA.1996.554977
- [5] Giorgio Buttazzo and Paolo Gai. 2006. Efficient EDF Implementation for Small Embedded Systems. In *2nd Workshop on Operating Systems Platforms for Embedded Real-Time Applications*. 7–9.
- [6] Cadence Design Systems, Inc. 2022. *Xtensa Instruction Set Architecture (ISA) Summary*. https://www.cadence.com/content/dam/cadence-www/global/en_US/documents/tools/silicon-solutions/compute-ip/isa-summary.pdf

- [7] Alessio Carlini and Giorgio C. Buttazzo. 2003. An Efficient Time Representation for Real-Time Embedded Systems. In *Proceedings of the 2003 ACM Symposium on Applied Computing (SAC), March 9-12, 2003, Melbourne, FL, USA*. ACM, 705–712. doi:10.1145/952532.952669
- [8] Antoine Colin and Isabelle Puaut. 2001. Worst-Case Execution Time Analysis of the RTEMS Real-Time Operating System. In *13th Euromicro Conference on Real-Time Systems (ECRTS 2001), 13-15 June 2001, Delft, The Netherlands, Proceedings*. IEEE Computer Society, 191–198. doi:10.1109/EMRTS.2001.934029
- [9] Arvind Easwaran, Insup Lee, Oleg Sokolsky, and Steve Vestal. 2009. A Compositional Scheduling Framework for Digital Avionics Systems. In *15th IEEE International Conference on Embedded and Real-Time Computing Systems and Applications, RTCSA 2009, Beijing, China, 24-26 August 2009*. IEEE Computer Society, 371–380. doi:10.1109/RTCSA.2009.46
- [10] Gurobi Optimization, LLC. 2024. *Gurobi Optimizer Reference Manual*. <https://www.gurobi.com>
- [11] Florian Hagens and Kuan-Hsun Chen. 2023. Assessment of Efficient Dispatching in FreeRTOS. In *18th Workshop on Operating Systems Platforms for Embedded Real-Time Applications (OSPERT)*. 7–9.
- [12] Wanja Hofer, Daniel Danner, Rainer Muller, Fabian Scheler, Wolfgang Schröder-Preikschat, and Daniel Lohmann. 2012. Sloth on Time: Efficient Hardware-Based Scheduling for Time-Triggered RTOS. In *Proceedings of the 33rd IEEE Real-Time Systems Symposium, RTSS*. 237–247. doi:10.1109/RTSS.2012.75
- [13] Fouad Khenfri, Khaled Chaaban, and Maryline Chetto. 2020. Efficient mapping of runnables to tasks for embedded AUTOSAR applications. *J. Syst. Archit.* 110 (2020), 101800. doi:10.1016/j.jsysarc.2020.101800
- [14] Simon Kramer, Dirk Ziegenbein, and Arne Hamann. 2015. Real world automotive benchmarks for free. In *6th International Workshop on Analysis Tools and Methodologies for Embedded and Real-time Systems*.
- [15] Tei-Wei Kuo and Aloysius K. Mok. 1991. Load Adjustment in Adaptive Real-Time Systems. In *Proceedings of the Real-Time Systems Symposium - 1991, San Antonio, Texas, USA, December 1991*. IEEE Computer Society, 160–170. doi:10.1109/REAL.1991.160369
- [16] Jaewoo Lee, Sisu Xi, Sanjian Chen, Linh T. X. Phan, Christopher D. Gill, Insup Lee, Chenyang Lu, and Oleg Sokolsky. 2012. Realizing Compositional Scheduling through Virtualization. In *2012 IEEE 18th Real Time and Embedded Technology and Applications Symposium, Beijing, China, April 16-19, 2012*, Marco Di Natale (Ed.). IEEE Computer Society, 13–22. doi:10.1109/RTAS.2012.20
- [17] Peter Marwedel. 2021. *System Software*. Springer International Publishing, Cham, 203–237. doi:10.1007/978-3-030-60910-8_4
- [18] Aloysius K. Mok. 1988. Task Management Techniques for Enforcing EDF Scheduling on a Periodic Task Set. In *5th IEEE Workshop on Real-Time Software and Operating Systems*. 42–46.
- [19] Mitra Nasri, Morteza Mohaqeqi, and Gerhard Fohler. 2016. Quantifying the Effect of Period Ratios on Schedulability of Rate Monotonic. In *Proceedings of the 24th International Conference on Real-Time Networks and Systems, RTNS 2016, Brest, France, October 19-21, 2016*, Alain Plantec, Frank Singhoff, Sébastien Faucou, and Luís Miguel Pinho (Eds.). ACM, 161–170. doi:10.1145/2997465.2997483
- [20] Gessé Oliveira and George Lima. 2023. Scheduling and energy savings for small scale embedded FreeRTOS-based real-time systems. *Des. Autom. Embed. Syst.* 27, 1-2 (2023), 3–29. doi:10.1007/S10617-023-09267-7
- [21] Risat Mahmud Pathan. 2015. Unifying fixed- and dynamic-priority scheduling based on priority promotion and an improved ready queue management technique. In *21st IEEE Real-Time and Embedded Technology and Applications Symposium, Seattle, WA, USA, April 13-16, 2015*. IEEE Computer Society, 209–220. doi:10.1109/RTAS.2015.7108444
- [22] Risat Mahmud Pathan. 2016. Design of an efficient ready queue for earliest-deadline-first (EDF) scheduler. In *2016 Design, Automation & Test in Europe Conference & Exhibition, DATE 2016, Dresden, Germany, March 14-18, 2016*. IEEE, 293–296. <https://ieeexplore.ieee.org/document/7459325/>
- [23] RTEMS Project and contributors. 2025. *RTEMS User Manual* (release 6.1 (22nd january 2025) ed.). On-Line Applications Research (OAR) Corporation. <https://docs.rtems.org/docs/6.1/user.pdf>
- [24] Richard Barry and The FreeRTOS Team. 2024. *Mastering the FreeRTOS Real Time Kernel - A Hands-On Tutorial Guide* (release version - 1.1.0 ed.). Amazon Web Services. <https://github.com/FreeRTOS/FreeRTOS-Kernel-Book/releases/download/V1.1.0/Mastering-the-FreeRTOS-Real-Time-Kernel.v1.1.0.pdf>
- [25] Michael Short. 2010. Improved Task Management Techniques for Enforcing EDF Scheduling on Recurring Tasks. In *16th IEEE Real-Time and Embedded Technology and Applications Symposium, RTAS 2010, Stockholm, Sweden, April 12-15, 2010*, Marco Caccamo (Ed.). IEEE Computer Society, 56–65. doi:10.1109/RTAS.2010.22
- [26] Espressif Systems. 2024. *ESP32-S3 Technical Reference Manual*. https://www.espressif.com/sites/default/files/documentation/esp32-s3_technical_reference_manual_en.pdf
- [27] George Varghese and Anthony Lauck. 1997. Hashed and hierarchical timing wheels: efficient data structures for implementing a timer facility. *IEEE/ACM Trans. Netw.* 5, 6 (1997), 824–834. doi:10.1109/90.650142
- [28] Georg von der Brüggen, Niklas Ueter, Jian-Jia Chen, and Matthias Freier. 2017. Parametric utilization bounds for implicit-deadline periodic task in automotive systems. In *Proceedings of the 25th International Conference on Real-Time Networks and Systems, RTNS 2017, Grenoble, France, October 04 - 06, 2017*. ACM, 108–117. doi:10.1145/3139258.3139273